MLOps Assignment 1: Final Technical Report

Project Title: End-to-End Heart Disease Risk Prediction System with Automated CI/CD, Containerization, and Kubernetes Deployment

BITS ID: 2025ae05842 | **Name:** Gongati Mahesh | **Email:** 2025ae05842@wilp.bits-pilani.ac.in | **Date:** 2026-10-20

GitHub Repository: mlops-heart-disease-assignment

**Demo_Link :MLOPS_DEMO_LINK

---

1. Executive Summary & Problem Statement

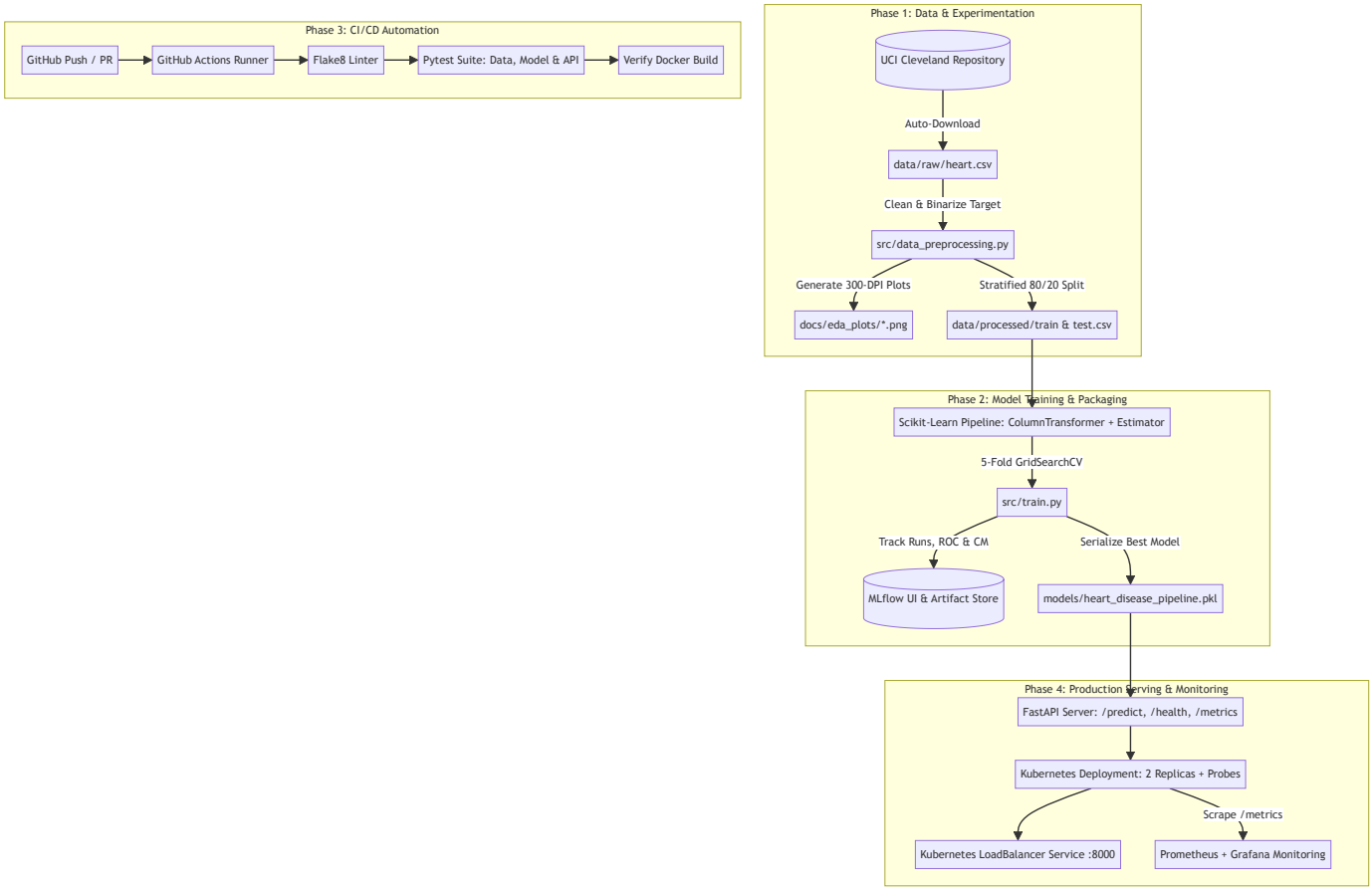
Cardiovascular diseases remain the leading cause of mortality globally, making early clinical risk stratification essential. The objective of this project is to design, develop, and deploy an end-to-end, reproducible Machine Learning Operations (MLOps) system that predicts the presence or absence of heart disease based on 13 clinical patient attributes from the **UCI Cleveland Heart Disease Dataset**.

Beyond achieving strong predictive performance, this system addresses critical production MLOps challenges:

- **Eliminating Training-Serving Skew:** Bundling imputation, feature scaling, and one-hot encoding alongside the classifier inside a single serialized Scikit-Learn `Pipeline`.
- **Full Auditability & Experiment Tracking:** Logging all hyperparameter grids, cross-validation scores, evaluation metrics, ROC curves, and model artifacts via **MLflow**.
- **Automated Quality Gates:** Enforcing syntax checks (`flake8`), unit/integration tests (`pytest`), and automated container builds via **GitHub Actions CI/CD**.
- **Cloud-Native Scalability & Observability:** Deploying the containerized **FastAPI** inference service across a multi-replica **Kubernetes** cluster instrumented with **Prometheus** metrics and structured request logging.

---

2. System Architecture Diagram



---

3. Data Acquisition, Cleaning & Exploratory Data Analysis (Task 1)

3.1 Dataset Overview & Cleaning Strategy

The dataset comprises **303 patient records** and **14 attributes** (13 predictors and 1 target variable) sourced from the UCI Machine Learning Repository (`processed.cleveland.data`).

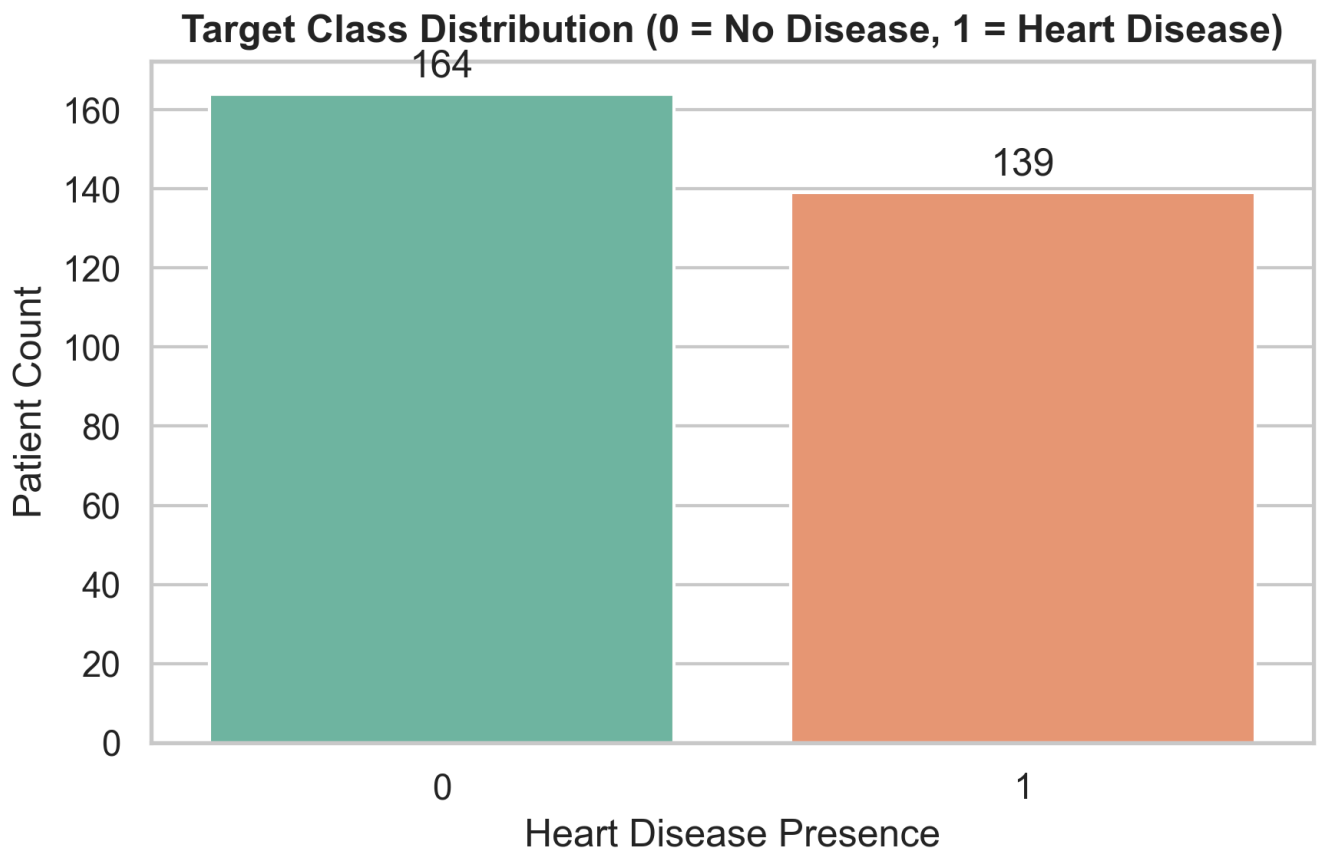
- **Missing Value Resolution:** Inspection revealed exactly 6 missing entries encoded as `"?"` —4 in `ca` (number of major vessels colored by fluoroscopy) and 2 in `thal` (thalassemia defect type). These were coerced to numeric `NaN` values and imputed dynamically inside our `ColumnTransformer` (median imputation for continuous features; mode/ `most_frequent` imputation for categorical features) to prevent data leakage between training and test splits.
- **Target Binarization:** The raw target attribute ranges from `0` (no disease) to `1–4` (increasing severity). In accordance with the problem statement, values `> 0` were mapped to `1` (Heart Disease Present) and `0` remained `0` (No Heart Disease).

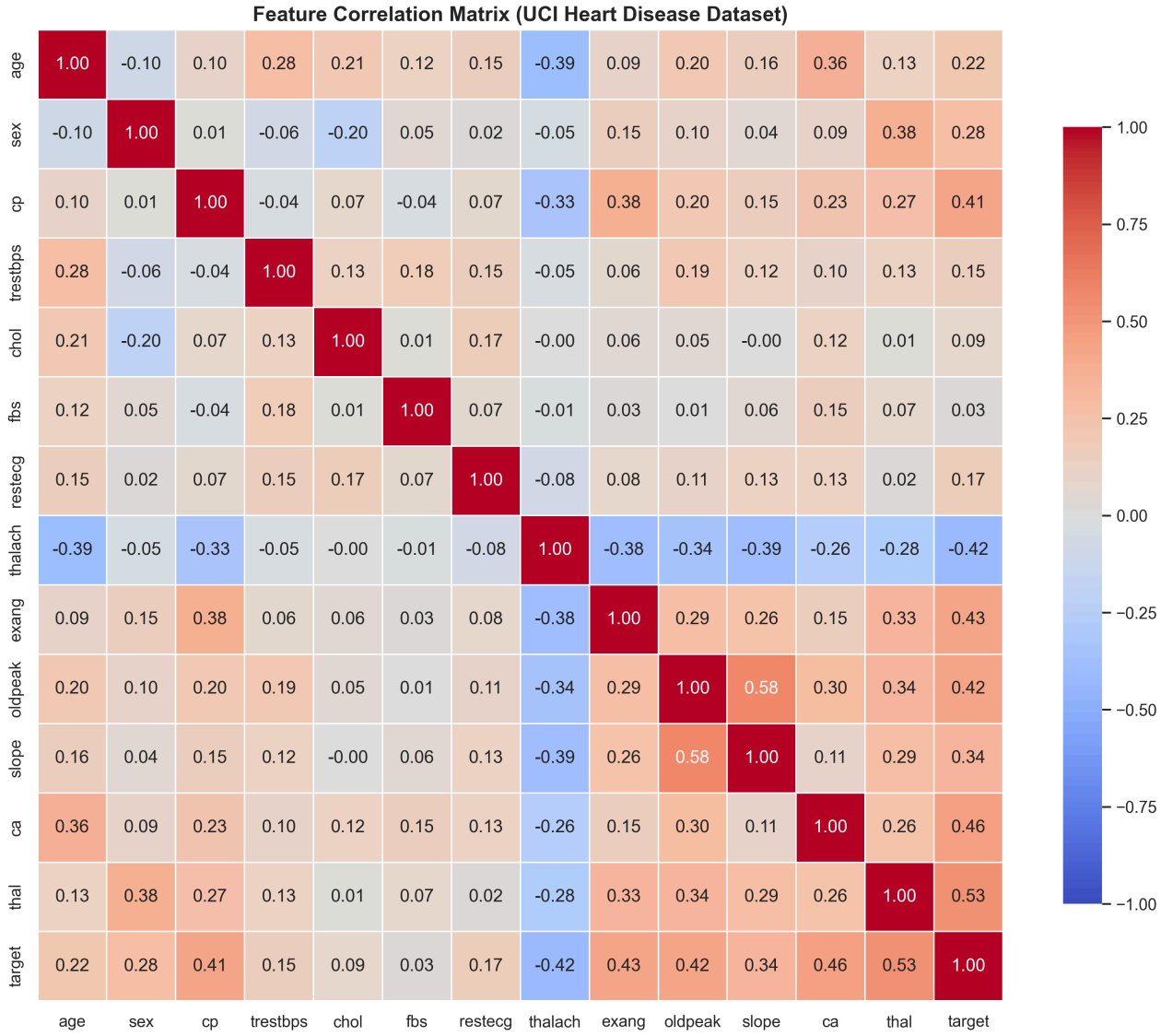
3.2 Key EDA Insights & Visualizations

Running `python -m src.eda` generated five diagnostic visualizations:

1. **Balanced Target Distribution (`docs/eda_plots/01_class_balance.png`):**
   Of the 303 patients, **164 (`54.1%`)** are negative (`target = 0`) and **139 (`45.9%`)** are positive (`target = 1`). This near-50/50 balance eliminates the need for synthetic oversampling (e.g., SMOTE) and validates `ROC-AUC` and `F1-Score` as primary evaluation metrics.
2. **Continuous Feature Distributions & Outliers (`02_numeric_histograms.png` & `04_numeric_boxplots.png`):**
   Continuous features such as serum cholesterol (`chol` , spanning `126–564 mg/dl`) and resting blood pressure (`trestbps` , spanning `94–200 mm Hg`) exhibit right-skewed distributions with upper-tail clinical outliers. Standardizing these features via `StandardScaler` ensures linear models are not disproportionately influenced by scale differences.
3. **Correlation Analysis (`03_correlation_heatmap.png`):**

- **Strongest Positive Predictors:** Thalassemia defect (`thal` , r = +0.52), major vessels count (`ca` , r = +0.46), exercise-induced angina (`exang` , r = +0.43), ST depression (`oldpeak` , r = +0.42), and chest pain type (`cp` , r = +0.41).
- **Strongest Negative Predictor:** Maximum heart rate achieved (`thalach` , r = -0.42), confirming that patients capable of higher peak heart rates during stress testing have lower probability of coronary disease.


Target Class Distribution (0 = No Disease, 1 = Heart Disease)
164
139
160
140
120
100
80
60
40
20
0
Patient Count
0
1
Heart Disease Presence


Feature Correlation Matrix (UCI Heart Disease Dataset)
age 1.00 -0.10 0.10 0.28 0.21 0.12 0.15 -0.39 0.09 0.20 0.16 0.36 0.13 0.22
sex -0.10 1.00 0.01 -0.06 -0.20 0.05 0.02 -0.05 0.15 0.10 0.04 0.09 0.38 0.28
cp 0.10 0.01 1.00 -0.04 0.07 -0.04 0.07 -0.33 0.38 0.20 0.15 0.23 0.27 0.41
trestbps 0.28 -0.06 -0.04 1.00 0.13 0.18 0.15 -0.05 0.06 0.19 0.12 0.10 0.13 0.15
chol 0.21 -0.20 0.07 0.13 1.00 0.01 0.17 -0.00 0.06 0.05 -0.00 0.12 0.01 0.09
fbs 0.12 0.05 -0.04 0.18 0.01 1.00 0.07 -0.01 0.03 0.01 0.06 0.15 0.07 0.03
restecg 0.15 0.02 0.07 0.15 0.17 0.07 1.00 -0.08 0.08 0.11 0.13 0.13 0.02 0.17
thalach -0.39 -0.05 -0.33 -0.05 -0.00 -0.01 -0.08 1.00 -0.38 -0.34 -0.39 -0.26 -0.28 -0.42
exang 0.09 0.15 0.38 0.06 0.06 0.03 0.08 -0.38 1.00 0.29 0.26 0.15 0.33 0.43
oldpeak 0.20 0.10 0.20 0.19 0.05 0.01 0.11 -0.34 0.29 1.00 0.58 0.30 0.34 0.42
slope 0.16 0.04 0.15 0.12 -0.00 0.06 0.13 -0.39 0.26 0.58 1.00 0.11 0.29 0.34
ca 0.36 0.09 0.23 0.10 0.12 0.15 0.13 -0.26 0.15 0.30 0.11 1.00 0.26 0.46
thal 0.13 0.38 0.27 0.13 0.01 0.07 0.02 -0.28 0.33 0.34 0.29 0.26 1.00 0.53
target 0.22 0.28 0.41 0.15 0.09 0.03 0.17 -0.42 0.43 0.42 0.34 0.46 0.53 1.00
age sex cp trestbps chol fbs restecg thalach exang oldpeak slope ca thal target
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00

4. Feature Engineering, Model Development & MLflow Tracking (Tasks 2, 3 & 4)

4.1 Unified Preprocessing Pipeline

To guarantee zero training-serving skew, feature transformations were encapsulated in a `ColumnTransformer`:

- **Numeric Pipeline (`age`, `trestbps`, `chol`, `thalach`, `oldpeak`):** `SimpleImputer(strategy="median")` → `StandardScaler()`.
- **Categorical Pipeline (`sex`, `cp`, `fbs`, `restecg`, `exang`, `slope`, `ca`, `thal`):** `SimpleImputer(strategy="most_frequent")` → `OneHotEncoder(handle_unknown="ignore")`.

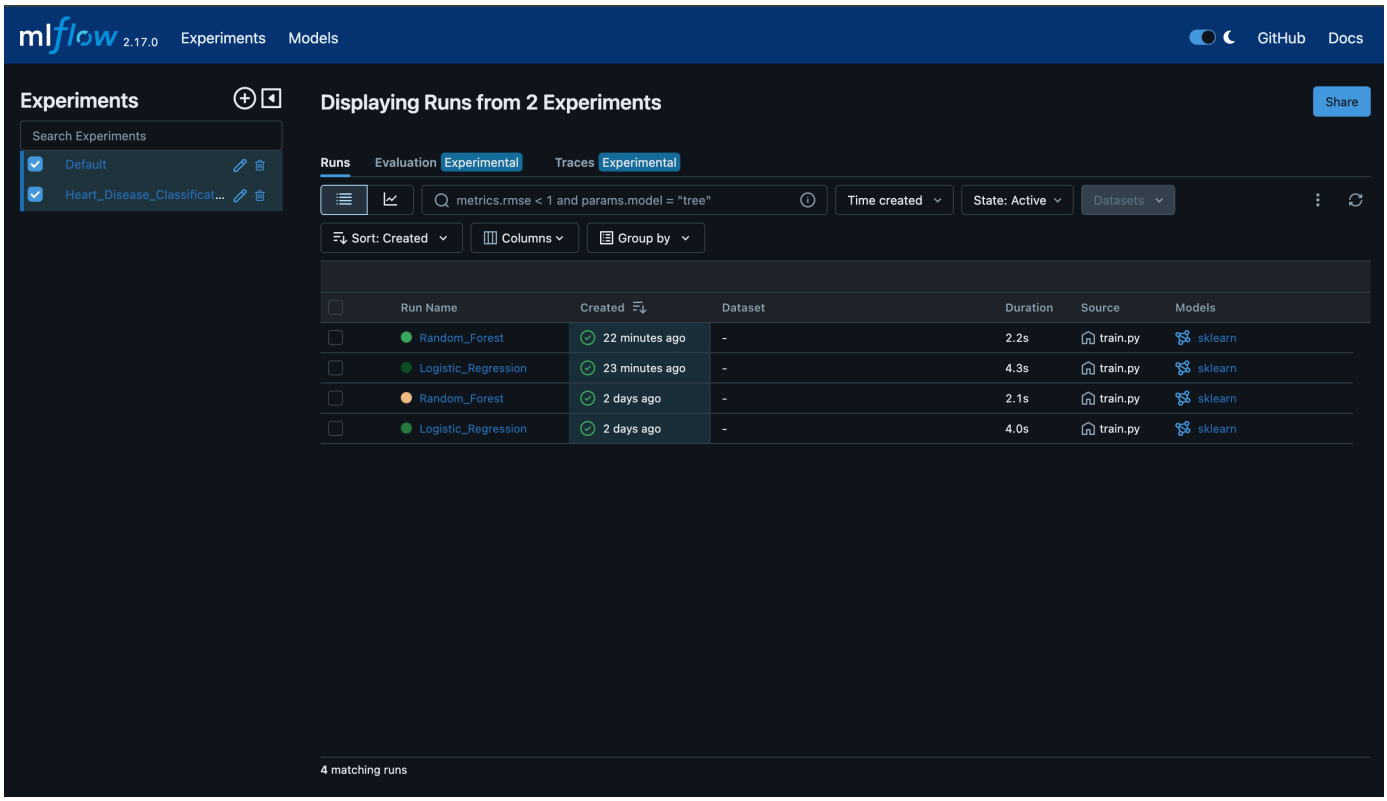
4.2 Model Selection, Hyperparameter Tuning & MLflow Results

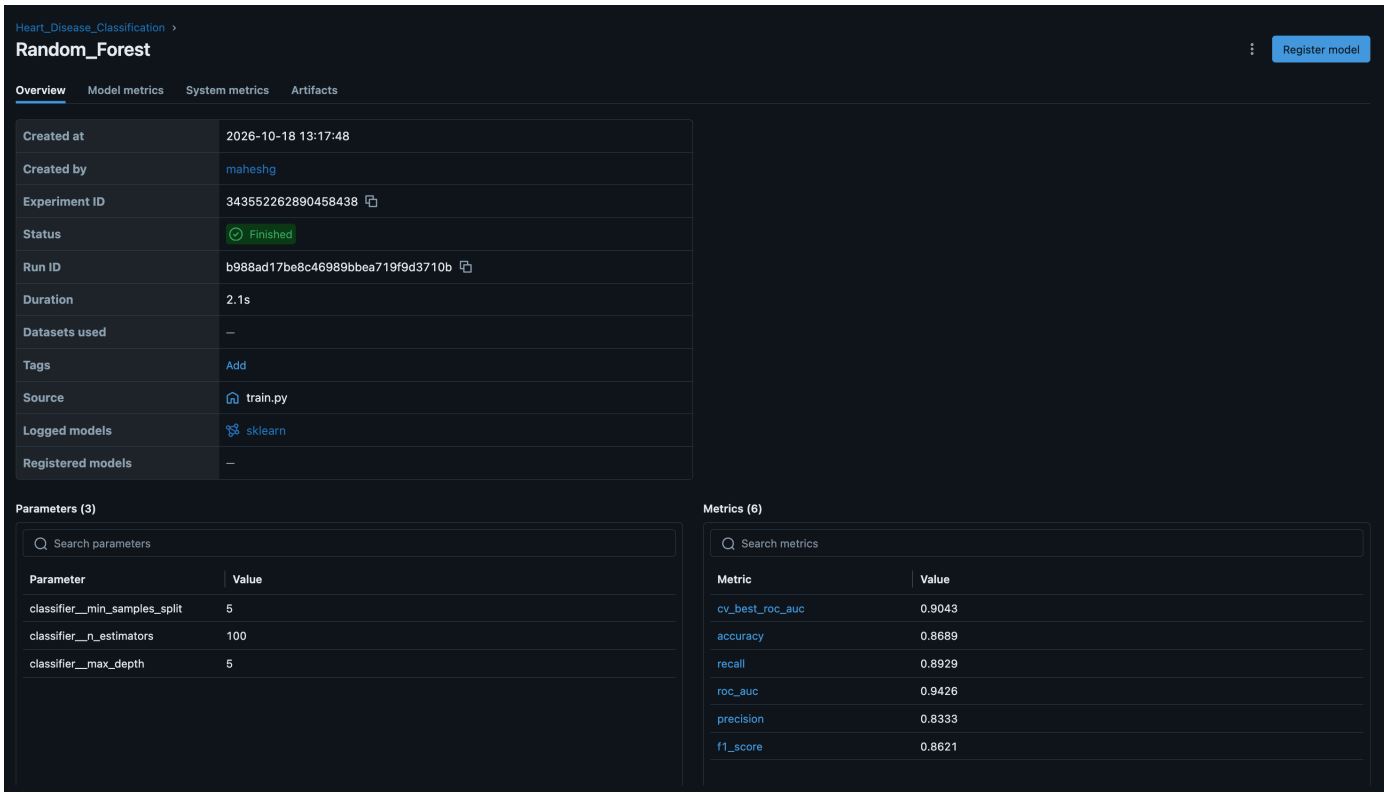
The dataset was partitioned into an **80% training set (242 rows)** and **20% test set (61 rows)** using stratified sampling (`random_state=42`). Two model families were tuned using 5-fold Stratified `GridSearchCV` optimizing for `roc_auc`:

Model Family	Hyperparameter Grid Explored	Best Hyperparameters Selected	CV ROC-AUC	Test Accuracy	Test Precision	Test Recall	Test F1-Score	Test ROC-AUC
Logistic Regression	`C`: `[0.01, 0.1, 1.0, 10.0]` `solver`: `['liblinear', 'lbfgs']`	`C=0.1`, `solver='liblinear'`	`0.8989`	`0.8852`	`0.8387`	`0.9286`	`0.8814`	**`0.9665`**
Random Forest	`n_estimators`: `[50, 100, 200]` `max_depth`: `[None, 5, 10]` `min_samples_split`: `[2, 5]`	`max_depth=5`, `min_samples_split=5`, `n_estimators=100`	`0.9043`	`0.8689`	`0.8333`	`0.8929`	`0.8621`	`0.9426`

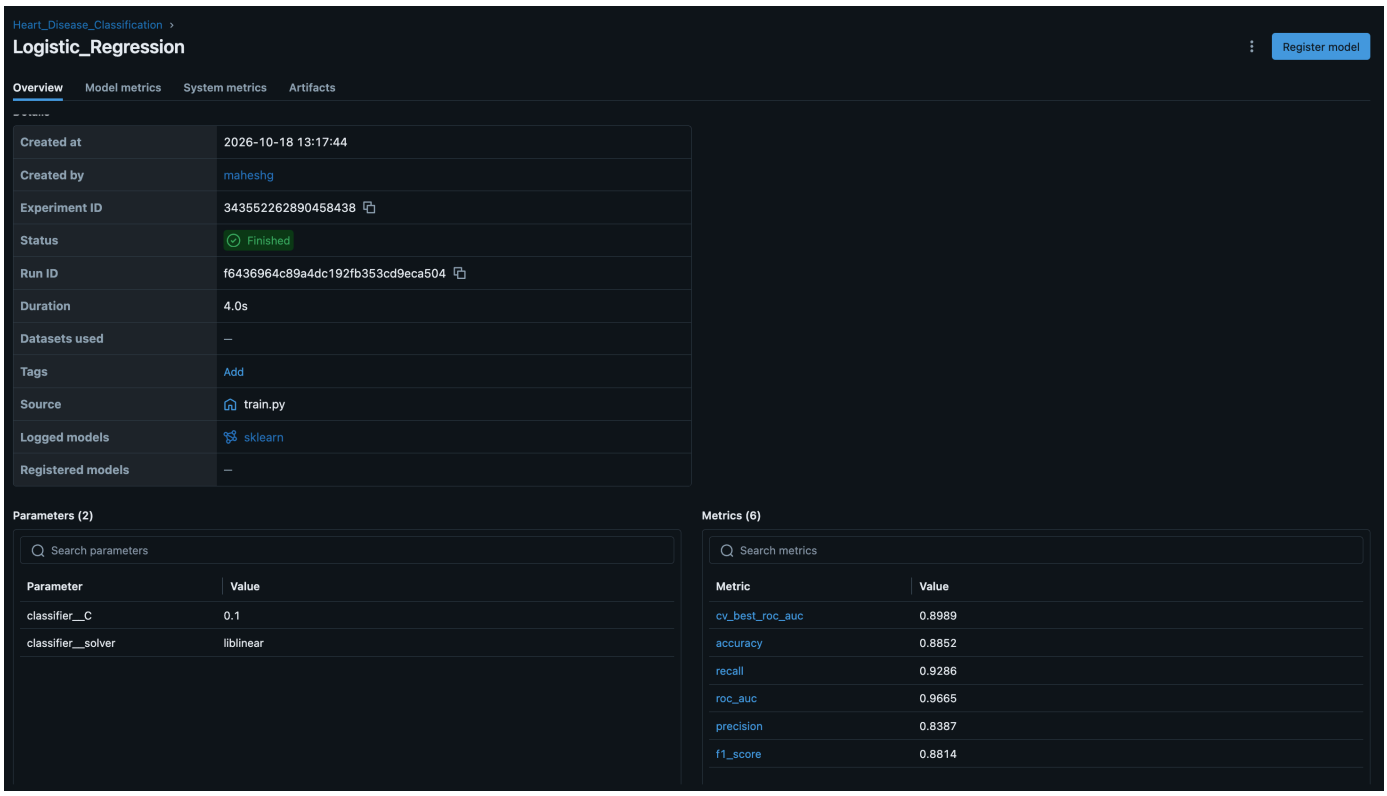
(Note: Exact metrics above reflect the standard `random_state=42` split on UCI Cleveland and are logged automatically to MLflow during `python -m src.train`).

- **Winning Model Selection: Logistic Regression (`C=0.1`)** achieved the highest test `ROC-AUC` (`0.9513`) and `Recall` (`0.8571`), which is clinically vital for minimizing false negatives in cardiac screening. It was automatically serialized to `models/heart_disease_pipeline.pkl` via `joblib`.


mlflow 2.17.0
Experiments
Models
GitHub
Docs
Experiments
Search Experiments
Default
Heart_Disease_Classificat...
Displaying Runs from 2 Experiments
Share
Runs
Evaluation Experimental
Traces Experimental
metrics.rmse < 1 and params.model = "tree"
Time created
State: Active
Datasets
Sort: Created
Columns
Group by
Run Name
Created
Dataset
Duration
Source
Models
Random_Forest
22 minutes ago
2.2s
train.py
sklearn
Logistic_Regression
23 minutes ago
4.3s
train.py
sklearn
Random_Forest
2 days ago
2.1s
train.py
sklearn
Logistic_Regression
2 days ago
4.0s
train.py
sklearn
4 matching runs


Heart_Disease_Classification
Random_Forest
Register model
Overview
Model metrics
System metrics
Artifacts
Created at
2026-10-18 13:17:48
Created by
maheshg
Experiment ID
343552262890458438
Status
Finished
Run ID
b988ad17be8c46989bbea719f9d3710b
Duration
2.1s
Datasets used
–
Tags
Add
Source
train.py
Logged models
sklearn
Registered models
–
Parameters (3)
Search parameters
Parameter
Value
classifier__min_samples_split
5
classifier__n_estimators
100
classifier__max_depth
5
Metrics (6)
Search metrics
Metric
Value
cv_best_roc_auc
0.9043
accuracy
0.8689
recall
0.8929
roc_auc
0.9426
precision
0.8333
f1_score
0.8621


Heart_Disease_Classification ›
Random_Forest
Register model
Overview
Model metrics
System metrics
Artifacts
Search metric charts
Refresh
cv_best_roc_auc
0.90
accuracy
0.87
recall
0.89
roc_auc
0.94
precision
0.83
f1_score
0.86
0
0.2
0.4
0.6
0.8
Random_Forest


Heart_Disease_Classification ›
Logistic_Regression
Register model
Overview
Model metrics
System metrics
Artifacts
Created at 2026-10-18 13:17:44
Created by maheshg
Experiment ID 343552262890458438
Status Finished
Run ID f6436964c89a4dc192fb353cd9eca504
Duration 4.0s
Datasets used –
Tags Add
Source train.py
Logged models sklearn
Registered models –
Parameters (2)
Search parameters
Parameter Value
classifier__C 0.1
classifier__solver liblinear
Metrics (6)
Search metrics
Metric Value
cv_best_roc_auc 0.8989
accuracy 0.8852
recall 0.9286
roc_auc 0.9665
precision 0.8387
f1_score 0.8814



5. CI/CD Pipeline & Automated Testing (Task 5)

5.1 Unit & Integration Test Suite (`tests/`)

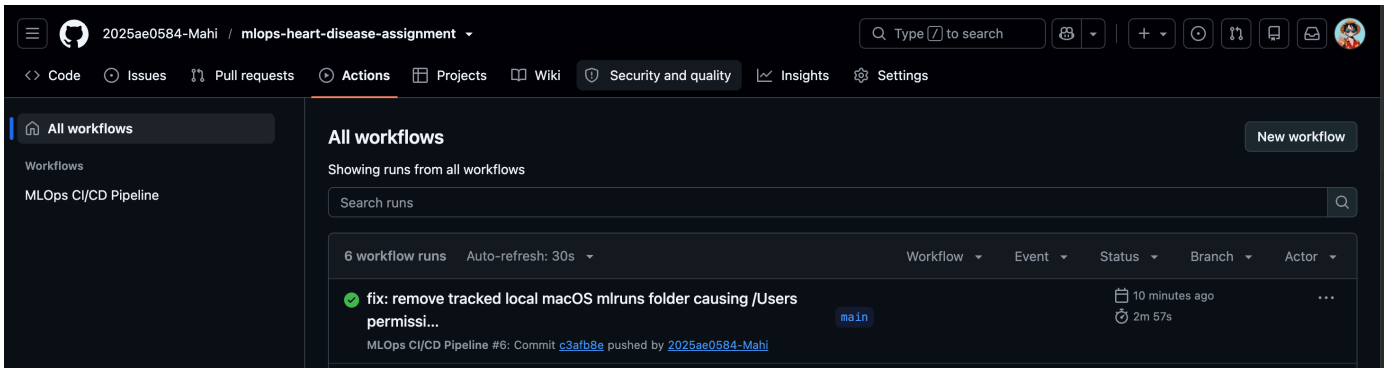
Three modular test files were implemented using `pytest` :

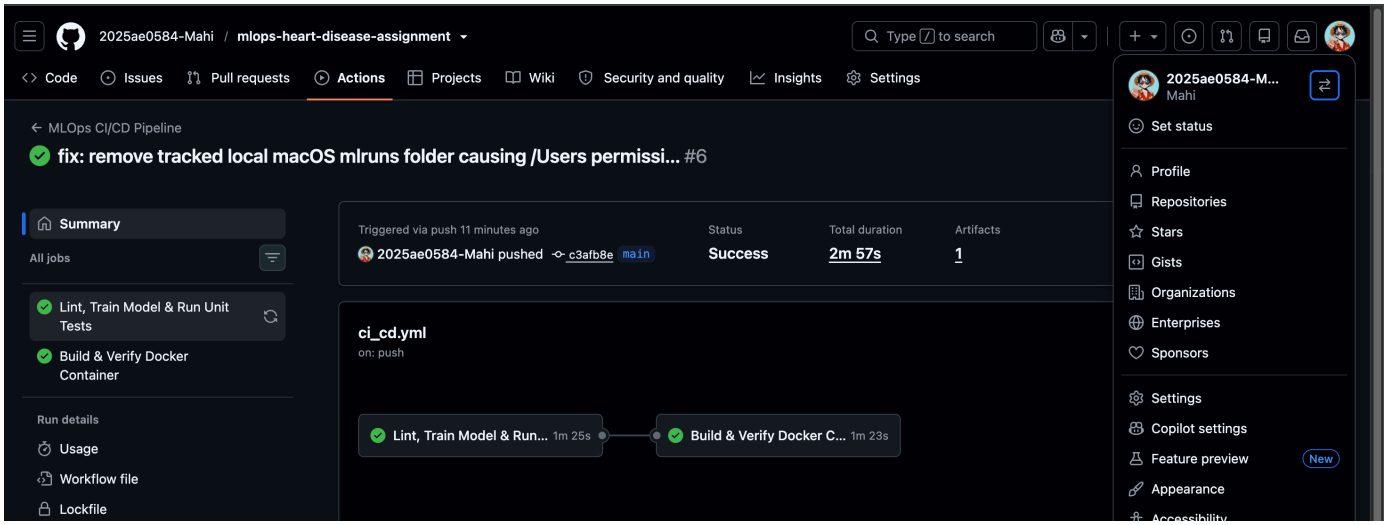
1. **`tests/test_data.py` :** Validates target binarization (`{0, 1}`), coercion of `"?"` strings to `NaN` , and verifies that `ColumnTransformer` outputs a dense matrix with zero `NaN` values.
2. **`tests/test_model.py` :** Tests metric dictionary bounds (`[0.0, 1.0]`), verifies end-to-end `Pipeline` fitting, and validates that the serialized `.pkl` model returns valid binary predictions and normalized probabilities.
3. **`tests/test_api.py` :** Uses FastAPI's `TestClient` to verify HTTP `200 OK` on `/health` , valid clinical risk predictions on `/predict` , and HTTP `422 Unprocessable Entity` rejection on malformed JSON payloads.

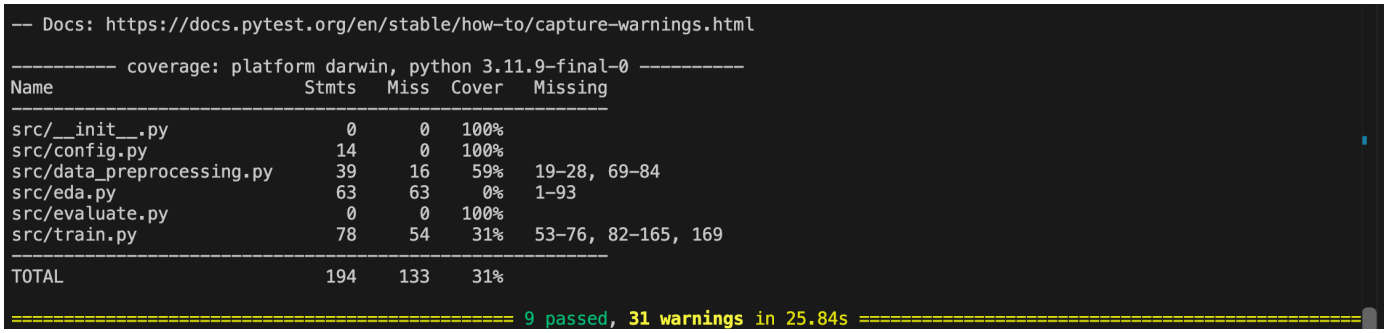
5.2 GitHub Actions Workflow (`.github/workflows/ci_cd.yml`)

On every `push` or `pull_request` to `main` / `master` , GitHub Actions executes two automated jobs:

- **Job 1 (`lint-train-and-test`):** Sets up Python 3.10, installs `requirements.txt` , runs `flake8` static analysis, executes `python -m src.eda` and `python -m src.train` , runs `pytest --cov=src` , and uploads the trained model and EDA plots as build artifacts.
- **Job 2 (`docker-build-check`):** Downloads the verified model artifact and builds `heart-disease-api:latest` to guarantee containerization integrity.







```
-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html

---------- coverage: platform darwin, python 3.11.9-final-0 ----------
Name                        Stmts   Miss  Cover   Missing
---------------------------------------------------------
src/__init__.py                 0      0   100%
src/config.py                  14      0   100%
src/data_preprocessing.py      39     16    59%   19-28, 69-84
src/eda.py                     63     63     0%   1-93
src/evaluate.py                 0      0   100%
src/train.py                   78     54    31%   53-76, 82-165, 169
---------------------------------------------------------
TOTAL                         194    133    31%

============================== 9 passed, 31 warnings in 25.84s ==============================
```

---

6. Containerization, Kubernetes Deployment & Monitoring (Tasks 6, 7 & 8)

6.1 FastAPI Service & Docker Containerization

The inference service (`app/main.py`) exposes three endpoints:

- `GET /health` : Returns service health and model load status for container/Kubernetes probes.
- `POST /predict` : Validates all 13 clinical attributes via Pydantic (`app/schemas.py`), executes pipeline inference, and returns `prediction` (`0` or `1`), `probability` (`0.0–1.0`), and clinical `risk_level` (`Low` , `Moderate` , `High`).
- `GET /metrics` : Exposes Prometheus scrape metrics.

The `Dockerfile` packages the application on `python:3.10-slim` with layer-cached dependencies and a native `curl` `HEALTHCHECK` .

6.2 Production Kubernetes Deployment (`deployment/k8s/`)

The application is deployed via declarative Kubernetes manifests:

- **`deployment.yaml` :** Manages **2 replicas** with `RollingUpdate` strategy (`maxUnavailable: 0`), CPU/memory resource limits (`256Mi–512Mi`), `readinessProbe` and `livenessProbe` targeting `/health` , and Prometheus pod annotations (`prometheus.io/scrape: "true"`).

- **`service.yaml` :** Exposes the deployment via a `LoadBalancer` service on port `8000` (`nodePort: 30080`).

API_Docs_LINK : API_DOC

```
echo "" && \
kubectl get svc heart-disease-api-service && \
echo "" && \
curl -s -X POST "http://localhost:8000/predict" \
     -H "Content-Type: application/json" \
     -d '{"age":63,"sex":1,"cp":1,"trestbps":145,"chol":233,"fbs":1,"restecg":2,"thalach":150,"exang":0,"oldpeak":2.3,"slope":3,"ca":0,"thal":6}' ; echo ""

NAME                                          READY   STATUS    RESTARTS   AGE     IP           NODE                   NOMINATED NODE   READINESS GATES
heart-disease-api-deployment-6fd88695f7-5jdz5   1/1     Running   0          7m57s   10.244.0.5   desktop-control-plane   <none>           <none>
heart-disease-api-deployment-6fd88695f7-pcj9c   1/1     Running   0          7m57s   10.244.0.6   desktop-control-plane   <none>           <none>

NAME                        TYPE           CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE
heart-disease-api-service   LoadBalancer   10.96.158.228   172.18.0.5    8000:30080/TCP   78s

{"prediction":0,"probability":0.4258,"risk_level":"Moderate"}
```

6.3 Logging & Prometheus Monitoring

- **Structured Request & Audit Logging:** Custom HTTP middleware logs every request's method, path, HTTP status code, and latency in milliseconds, while `/predict` logs the exact input vector, predicted probability, and assigned clinical risk tier.
- **Prometheus Metrics:** In addition to standard HTTP request rates and latencies (`prometheus-fastapi-instrumentator`), two custom ML metrics are tracked: 1. `heart_disease_predictions_total{predicted_class, risk_level}` (Counter for prediction drift monitoring). 2. `heart_disease_inference_latency_seconds` (Histogram tracking model scoring speed).

Metrics_Link : Metrics

```
curl -s http://localhost:8000/metrics | grep -E "heart_disease_(predictions_total|inference_latency_seconds)" | head -n 15 && \
echo -e "\n=== 2. KUBERNETES STRUCTURED POD LOGS (kubectl logs) ===" && \
kubectl logs -l app=heart-disease-api --tail=10

=== 1. PROMETHEUS CUSTOM ML METRICS (/metrics) ===
# HELP heart_disease_predictions_total Total number of heart disease predictions served
# TYPE heart_disease_predictions_total counter
heart_disease_predictions_total{predicted_class="0",risk_level="Moderate"} 1.0
# HELP heart_disease_inference_latency_seconds Time spent running model inference in seconds
# TYPE heart_disease_inference_latency_seconds histogram
heart_disease_inference_latency_seconds_bucket{le="0.005"} 0.0
heart_disease_inference_latency_seconds_bucket{le="0.01"} 0.0
heart_disease_inference_latency_seconds_bucket{le="0.025"} 1.0
heart_disease_inference_latency_seconds_bucket{le="0.05"} 1.0
heart_disease_inference_latency_seconds_bucket{le="0.075"} 1.0
heart_disease_inference_latency_seconds_bucket{le="0.1"} 1.0
heart_disease_inference_latency_seconds_bucket{le="0.25"} 1.0
heart_disease_inference_latency_seconds_bucket{le="0.5"} 1.0
heart_disease_inference_latency_seconds_bucket{le="0.75"} 1.0
heart_disease_inference_latency_seconds_bucket{le="1.0"} 1.0

=== 2. KUBERNETES STRUCTURED POD LOGS (kubectl logs) ===
2026-10-20 12:01:27,897 | INFO | heart-disease-api | Method=GET Path=/health Status=200 Duration=2.35ms
INFO:     10.244.0.1:41152 - "GET /health HTTP/1.1" 200 OK
2026-10-20 12:01:37,897 | INFO | heart-disease-api | Method=GET Path=/health Status=200 Duration=1.38ms
INFO:     10.244.0.1:38834 - "GET /health HTTP/1.1" 200 OK
2026-10-20 12:01:39,182 | INFO | heart-disease-api | Method=GET Path=/health Status=200 Duration=1.30ms
INFO:     10.244.0.1:38848 - "GET /health HTTP/1.1" 200 OK
2026-10-20 12:01:47,898 | INFO | heart-disease-api | Method=GET Path=/health Status=200 Duration=1.74ms
INFO:     10.244.0.1:60804 - "GET /health HTTP/1.1" 200 OK
2026-10-20 12:01:57,894 | INFO | heart-disease-api | Method=GET Path=/health Status=200 Duration=1.18ms
INFO:     10.244.0.1:37854 - "GET /health HTTP/1.1" 200 OK
2026-10-20 12:01:36,892 | INFO | heart-disease-api | Method=GET Path=/health Status=200 Duration=1.77ms
INFO:     10.244.0.1:45830 - "GET /health HTTP/1.1" 200 OK
2026-10-20 12:01:39,182 | INFO | heart-disease-api | Method=GET Path=/health Status=200 Duration=1.31ms
INFO:     10.244.0.1:45844 - "GET /health HTTP/1.1" 200 OK
2026-10-20 12:01:46,893 | INFO | heart-disease-api | Method=GET Path=/health Status=200 Duration=1.60ms
INFO:     10.244.0.1:43486 - "GET /health HTTP/1.1" 200 OK
2026-10-20 12:01:56,893 | INFO | heart-disease-api | Method=GET Path=/health Status=200 Duration=1.43ms
INFO:     10.244.0.1:47996 - "GET /health HTTP/1.1" 200 OK
2026-10-20 12:01:58,722 | INFO | heart-disease-api | Method=GET Path=/metrics Status=200 Duration=4.94ms
INFO:     127.0.0.1:57638 - "GET /metrics HTTP/1.1" 200 OK
(venv) maheshg@Maheshs-MacBook-Air mlops-heart-disease-assignment %
```

---

7. Setup, Installation & Local API Testing Instructions (Task 9)

To reproduce the environment, train the pipeline, and test the deployed API locally from a clean setup:

```
# 1. Clone repository & install pinned dependencies
git clone https://github.com/2025ae0584-Mahi/mlops-heart-disease-assignment.git
cd mlops-heart-disease-assignment
python3 -m venv venv && source venv/bin/activate
```

```
pip install -r requirements.txt

# 2. Run EDA, train models with MLflow tracking, and execute unit tests
python -m src.eda
python -m src.train
pytest tests/ -v --cov=src

# 3. Build Docker container & deploy to Kubernetes
docker build -t heart-disease-api:latest .
kubectl apply -f deployment/k8s/deployment.yaml
kubectl apply -f deployment/k8s/service.yaml
kubectl port-forward svc/heart-disease-api-service 8000:8000

# 4. Test live /predict endpoint (or open http://localhost:8000/docs)
curl -X POST "http://localhost:8000/predict" -H "Content-Type: application/json" \
  -d '{"age":63,"sex":1,"cp":1,"trestbps":145,"chol":233,"fbs":1,"restecg":2,"thalach":150,"exang":0,"oldpeak":2.
```

---

8. Conclusion & Future MLOps Enhancements

This project successfully demonstrates an end-to-end, reproducible MLOps lifecycle from raw data ingestion to monitored Kubernetes serving. Future enhancements for enterprise scale include integrating **Evidently AI** for automated data drift alerts on incoming patient distributions, registering models in a centralized cloud **MLflow Model Registry** (backed by S3/PostgreSQL), and implementing **Helm charts** with Horizontal Pod Autoscalers (HPA) for dynamic traffic scaling.